

Pre-External-Audit Findings — Lean Access-Domain Program

Internal review against the `access-domain-redesign` branch, program `v2.1.0`, deployed at `rHSumT63fgwAsHhKbR39AijjY28H99VuvM8xNaNehkj` on devnet.

Scope: every Rust source file under `programs/dawn/src/` — ~2,605 lines across 27 files. Off-chain sealing layer (`sim/codec/sealing.ts`) was reviewed but is out of scope for the on-chain audit and uses well-vetted primitives (`libsodium sealed-box` via `tweetnacl-sealedbox-js`, `Ed25519↔Curve25519` conversion via `@noble/curves`).

This document lists everything an external auditor will likely flag, with our recommendations and current status. The intent is that the team addresses HIGH-severity items before sending the program to the external firm; MEDIUM and below can be discussed during the audit.

1. Executive summary

No CRITICAL findings. The program has a small, well-organized attack surface: 16 instructions, 9 account types, all caller-gated, no CPI to user-controlled programs (only `SystemProgram` for account creation). Authorization is consistently enforced via Anchor `address = X.owner` constraints and explicit `require!` checks.

4 HIGH-severity findings — all are operational gaps rather than exploits:

- No way to rotate `AccessDomain.owner` once set (recovery problem).
- No way to revoke an `AuthMethod` (forever-growing account).
- Dead code in the unsigned-height comparison creates false confidence.
- `add_device_for` allows the caller to assign Device ownership to any pubkey without that pubkey's consent.

12 MEDIUM-severity findings, mostly hardening: missing range validations, unenforced cross-account references, dead code, single-key authority for protocol-wide ops.

4 LOW / 2 INFO items for completeness.

2. Methodology

The review walked every Rust file in `programs/dawn/src/` looking for:

1. **Authorization paths** — every instruction's signer constraint and the explicit checks inside the handler.
2. **PDA seed correctness** — whether seeds are unique, stable across schema versions, and resistant to collision/preimage manipulation.
3. **Cross-account references** — whether each `Pubkey` field on an account is verified to point at a valid account of the right type when used.
4. **Input validation** — bounds, length caps, non-default-pubkey checks, range checks on numerics.
5. **State transitions** — close-recreate semantics, `init` vs. `init_if_needed`, account-already-initialized markers.
6. **Cryptographic correctness** — the on-chain program treats the `sealed_payload` as opaque, so the audit reduces to "is the buffer stored without modification?" (yes).
7. **Operational concerns** — upgrade authority handling, recovery paths, timelock, multisig recommendations.
8. **Dead code** — anything an auditor will ask "why is this here?"

Cross-referenced against the access-domain redesign rationale doc (`docs/account-schema-rationale.md`) to confirm intended behavior matches implementation.

3. Findings

Severity rubric:

- **CRITICAL** — exploit possible, fix before any further deployment.
- **HIGH** — must fix before mainnet. Operationally significant or externally observable as wrong.
- **MEDIUM** — should fix; auditors will require an explanation.
- **LOW** — defensive hardening; auditors will mention but won't block.
- **INFO** — context the auditor should know.

3.1 HIGH — H-01: No way to rotate `AccessDomain.owner`

Location: `programs/dawn/src/app/access_domain/` — instruction is absent.

Description: Once an AccessDomain is created, its `owner` field is fixed forever. Available `set_*` instructions only mutate `control_plane_device` and `gateway_device`. There is no `set_access_domain_owner` instruction.

Impact: If the operator's cold-admin key is compromised or rotated out of cold storage, **every AccessDomain owned by that key becomes unusable in terms of permission delegation**. The compromised key can still grant Tier-2 roles to attackers, and the legitimate operator cannot regain control without abandoning every AD they own and re-creating fresh ones. Existing customers' Credentials are still valid (they decrypt off-chain), but the operator can no longer manage the AD's auth methods, registrar grants, or device pointers.

Recommendation: Add `set_access_domain_owner(new_owner: Pubkey)`, gated on `address = access_domain.owner`. Reject default Pubkey. Emit `AccessDomainOwnerRotated{previous, new, updated_at}` event for audit. Document that this is a destination-side operation — the new owner takes over completely; existing DomainAuthority grants survive.

3.2 HIGH — H-02: No way to revoke an AuthMethod

Location: `programs/dawn/src/app/amf/` — instruction is absent.

Description: `register_auth_method` creates an AuthMethod PDA via Anchor `init`. There is no corresponding `revoke_auth_method` close instruction. Once created, an AuthMethod can have its parameters mutated in place via `update_auth_method_params`, but it can never be removed.

Impact: AccessDomain accumulates AuthMethod accounts indefinitely. Operationally minor today (one per (AD, method_type), max 6 per AD even with the future EAP/IPsec/WPA expansion). But:

- Rent for stale AuthMethod accounts is locked forever from the operator's wallet.
- An operator deprecating a network can't clean up; the AD stays visible on-chain.
- No path to "fully decommission" an AccessDomain without leaving AuthMethod orphans.

Recommendation: Add `revoke_auth_method`, gated on `address = access_domain.owner`. `close = caller` to reclaim rent. Reject if any active Credentials reference the AuthMethod (require the operator to revoke creds first). Emit `AuthMethodRevoked` event.

3.3 HIGH — H-03: Dead-code unsigned comparison in height validation

Location: `programs/dawn/src/app/device/add_device.rs:104` and `add_device_for.rs:99`:

```
require!(height.eq(&0u16), DawnError::InvalidHeight);
require!(height.lt(&0u16), DawnError::InvalidHeight); // ← always true
```

Description: `height: u16` is an unsigned 16-bit integer; it can never be less than zero. The `lt` check is trivially `true`. The intent was presumably an upper-bound or non-negativity guard, but as written, the second `require!` is dead code.

Impact: False sense of validation. An auditor will flag this. Code reviewers may believe the check enforces something it doesn't.

Recommendation: Either delete the redundant check, or replace with a meaningful upper-bound (e.g., `require!(height <= MAX_HEIGHT_M, ...)` where `MAX_HEIGHT_M` is, say, 100m for an AP mast). Same fix in both `add_device.rs` and `add_device_for.rs`.

3.4 HIGH — H-04: `add_device_for` allows unilateral Device assignment

Location: `programs/dawn/src/app/device/add_device_for.rs` — the `beneficiary: AccountInfo<'info>` is *not* required to sign.

Description: Any caller can submit `add_device_for` with any pubkey as `beneficiary`, and the resulting Device + LocalDomain will have that pubkey as `owner`. The beneficiary has no on-chain acknowledgement. The caller pays rent.

Intended use case: an operator registers an AAA appliance Device on behalf of a SoT-bridge wallet at provisioning time. This is the core flow for the bridge integration and is a legitimate pattern.

Issues:

1. The beneficiary inherits a `LocalDomain` they may not know about. If they ever try to register their own LocalDomain at the same name, the seeds collide.
2. The beneficiary's wallet "owns" Device accounts they cannot reclaim rent from until they realize the accounts exist.
3. An attacker could spam-create Devices targeting popular wallet pubkeys, polluting their on-chain account set. Costs the attacker rent (~0.005 SOL per Device), so not exploitable for free, but a nuisance.

Recommendation: Two options, in order of preference:

- (a) Require beneficiary to sign. This is the cleanest; the beneficiary explicitly opts in. Keypairs can be co-signed via a multi-sig tx. Breaks the convenience of "operator pre-provisions before bridge is up", though.
- (b) Add a co-signing step: caller submits `add_device_for`, beneficiary later submits `accept_device_assignment` which sets an `accepted: bool` flag on the Device. Until accepted, the Device is in a quarantined state (not usable as `control_plane_device`).

For the immediate use case, (b) preserves operator-side pre-provisioning while preventing wallet-pollution. Auditors will prefer (a).

3.5 MEDIUM — M-01: AccessDomain field references unverified

Location: `programs/dawn/src/app/access_domain/add_access_domain.rs` ,
`set_control_plane_device.rs` , `set_gateway_device.rs` .

Description: `control_plane_device` , `gateway_device` , and `local_domain` are stored as raw `Pubkey` fields on the `AccessDomain`. The program does NOT verify these point at actual `Device` / `LocalDomain` PDAs. Any pubkey is accepted (with the only constraint being that `control_plane_device != Pubkey::default()`).

Intended: This is by design — the redesign's contract is that `control_plane_device` is "any pubkey the operator wants the encryption recipient derived from." It does not have to be a `Device` PDA owned by the program.

Risk: An operator could mistakenly point at the wrong pubkey, breaking decryption silently (no on-chain validation catches it). The bridge will fail to decrypt and emit a runtime error, which is detectable but late.

Recommendation: Either:

- (a) Add an optional account-context validation: pass the actual `Device/LocalDomain` accounts to `add_access_domain` and verify they exist + their seeds match. Rejects garbage pubkeys at AD creation. Increases account count by up to 3 for one tx but provides strong guarantees.
- (b) Document explicitly that these fields are "any pubkey" with the stated semantics. The audit firm will accept either.

3.6 MEDIUM — M-02: `qos_tag` has no range validation

Location: `programs/dawn/src/app/amf/register_credential_for.rs` and `state/credential.rs` .

Description: `qos_tag: Option<u8>` accepts any value 0..255. The account doc says "convention (DSCP / 802.1p) is operator-side." For DSCP, valid range is 0..63 (6-bit field). For 802.1p, 0..7 (3-bit field).

Risk: An operator-api with a coding error could write a `qos_tag` like 200 that no real QoS scheme accepts. The bridge would forward it to RADIUS, which would either reject the row or apply a wrong QoS class.

Recommendation: Either validate to 0..63 (DSCP, the more permissive) or document the operator-side convention prominently with `#[derive(...)]` field doc and the `docs/sot-bridge-protocol.md` already addresses this. If validating, this is a one-line require! in the `register_credential*` paths.

3.7 MEDIUM — M-03: `created_at == 0` is a fragile init marker

Location: `add_device.rs:144`, `add_device_for.rs:140`:

```
if ctx.accounts.local_domain.created_at == 0 {  
    // freshly initialized – set fields  
}
```

Description: With Anchor's `init_if_needed`, the program needs to detect whether the `LocalDomain` account was just created vs. already existed. The current code uses `created_at == 0` as the marker. `created_at` is `i64` and is set to `Clock::get()?.unix_timestamp` on first init.

Risk: Anchor's `init_if_needed` initializes the account's data buffer to zero before deserializing. So a freshly-allocated `LocalDomain` has `created_at == 0`. This works in practice. But:

- If a future schema change moves `created_at` to a different offset or changes its type, the marker breaks silently.
- The pattern relies on knowing Anchor's internal init behavior.
- Auditors prefer explicit "was this just initialized?" markers.

Recommendation: Use `bump == 0` as the marker (a real PDA's bump is in 0..255 inclusive, but bump 0 is exceedingly rare and would imply a hash-grinded address — practically never accidentally collides). Better still: refactor to two instructions (`add_local_domain` separately from `add_device`) so the init pattern is explicit. Last option: use Anchor's `#[account(init, ...)]` constraint with two separate instructions (simpler design, slightly different UX).

3.8 MEDIUM — M-04: `verify_device_location` has no inverse

Location: `programs/dawn/src/app/device/verify_device_location.rs`.

Description: Once a `DeviceLocation` is verified by `Config.authority`, there's no instruction to un-verify it. If the operator misverifies (e.g., the device later moves, or the verification was issued to the wrong device), the verified flag stays true forever.

Recommendation: Add `unverify_device_location` (gated on `Config.authority`) that sets `verified = false` and clears `verified_at`. Emit `DeviceLocationUnverified` event for audit trail.

3.9 MEDIUM — M-05: Dead code in utils/

Location:

- `utils/seeds.rs` — `optional_pubkey_seed` has zero callers in the lean program.
- `utils/seeds.rs` — `_optional_i64_seed` is leading-underscored (intentionally dead) but still present.
- `utils/hash.rs` — `hash_parameters` has zero callers in the lean program.
- `utils/hash.rs` — `canonicalize_string_seed` is `#[deprecated]` with `#[allow(dead_code)]`.

The build emits `warning: function 'optional_pubkey_seed' is never used` on every clean build (visible in cargo output).

Risk: Auditors scrutinize unused helpers ("why does this function exist? does it imply dropped functionality? is it a hidden code path?"). Dead code increases audit cost. Also:

`canonicalize_string_seed` returns `&[u8]` directly without trimming or hashing — historically used with PDA seeds, now superseded by `hash_string_seed`. If a contributor accidentally uses it, they'd introduce a seed collision risk.

Recommendation: Delete `optional_pubkey_seed`, `_optional_i64_seed`, `hash_parameters`, and `canonicalize_string_seed` before sending to audit. Trivial to revert via git history if needed later.

3.10 MEDIUM — M-06: Enum byte-stability has no compile-time guard

Location: `state/auth_method_type.rs` and `state/domain_authority.rs`.

Description: Both `AuthMethodType::as_seed()` and `DomainAuthorityRole::as_seed()` return single-byte arrays whose values are part of the PDA derivation. Documentation says "stable across schema versions: never reuse a byte for a different variant."

Risk: The promise is enforced only by developer discipline. A contributor reordering enum variants or inserting a new variant in the middle would silently change the byte mapping, invalidating every existing PDA.

Recommendation: Add a Rust unit test that asserts each variant's byte value:

```
#[test]
fn auth_method_type_seeds_stable() {
    assert_eq!(AuthMethodType::Psk.as_seed(), &[0]);
    assert_eq!(AuthMethodType::Mpsk.as_seed(), &[1]);
}
```

Same for `DomainAuthorityRole`. Future contributors who change the mapping will break the test, forcing review.

3.11 MEDIUM — M-07: `Config.authority` rotation is single-key, no timelock

Location: `app/update_config.rs`.

Description: `update_config_authority` allows the current authority to instantly rotate to a new authority with a single signature. No multisig requirement, no timelock.

Risk: Compromised cold-admin key → instant rotation to attacker. Attacker now controls all `Config`-gated operations (`add_device_model`, `verify_device_location`).

Mitigation today: This is a wallet-level concern, addressable by deploying with the cold-admin key being a multisig (Squads or similar). The contract doesn't need to know it's a multisig.

Recommendation: Document in the deployment runbook that mainnet deployment must:

- Set `Config.authority` to a multisig wallet (recommended: Squads with $\geq 2/3$ signers).
- Set program upgrade authority to the same multisig OR `None` for immutability.

If a contract-level timelock is desired, add a `TimelockedAuthority` pattern:

`update_config_authority(new)` queues the rotation; `finalize_config_authority_rotation()` applies it after a configurable delay (e.g., 48h). Out of scope for v2.1.0; track for v2.2.0.

3.12 MEDIUM — M-08: No emergency-pause mechanism

Location: No instruction provides this.

Description: If a vulnerability is discovered post-mainnet, there is no way to halt new credential mints, `AccessDomain` creations, etc., short of a program upgrade.

Recommendation: Add a `Config.paused: bool` field and `pause()` / `unpause()` instructions gated on `Config.authority`. Sensitive instructions (`register_credential_for`, `register_auth_method`, `grant_domain_authority_*`) check `!config.paused` at the start of the handler. Track for v2.2.0.

3.13 MEDIUM — M-09: AccessDomain seed uses caller, not stored owner

Location: `add_access_domain.rs:30:`

```
seeds = [  
    AccessDomain::SEED_PREFIX,  
    caller.key().as_ref(),  
    &hash_string_seed(&name),  
],
```

vs. `set_control_plane_device.rs:18:`

```
seeds = [  
    AccessDomain::SEED_PREFIX,  
    access_domain.owner.as_ref(),  
    &hash_string_seed(&access_domain.name),  
],
```

Description: Creation uses `caller.key()` ; lookup uses `access_domain.owner` . These are equivalent at creation time (`access_domain.owner = caller.key()`). The `set_*` paths use the stored owner, which is correct.

Risk: None — this is an aesthetic/consistency observation. However, if `set_access_domain_owner` is ever added (per H-01), the stored owner could diverge from `caller.key()` , and the create-site seed pattern would no longer be reusable for re-derivation by other wallets. Document this contract.

Recommendation: No code change required. Ensure that any future `set_access_domain_owner` instruction correctly handles the seed re-derivation by the NEW owner — actually, since the PDA is fixed once created, the seed correspondence simply stops being meaningful post-rotation. The PDA continues to work via `bump` lookups; the seeds become historical artifacts.

3.14 MEDIUM — M-10: Two PSK variants but only one validation path

Location: `app/amf/register_auth_method.rs:85` and `psk_method.rs::PSKMethodParams::validate()` .

Description: `validate_auth_params` treats `Psk` and `Mpsk` identically, validating both against `PSKMethodParams` . But semantically, single-PSK and multi-PSK have different meanings: single-PSK is one PSK shared by all customers (no Credential needed); multi-PSK is per-customer PSK delivered via Credential.

The current code never enforces that a `Psk AuthMethod` has at most one band populated (a single-PSK SSID typically broadcasts one band) or that an `Mpsk AuthMethod` has the per-customer aspect declared elsewhere.

Risk: Operators could create a `Psk AuthMethod` and then mint many Credentials against it, contradicting the "single PSK" semantic. The on-chain program won't reject this. Off-chain consumers will behave inconsistently.

Recommendation: Either:

- Enforce semantic difference (`Psk` allows zero Credentials; `Mpsk` allows N).
- Document in `psk_method.rs` that the on-chain program treats `Psk` and `Mpsk` identically and the distinction is purely advisory for off-chain consumers.

The lean implementation appears to take the latter view (they share `PSKMethodParams`). State this explicitly.

3.15 LOW — L-01: No defensive check on `sealed_payload`

Location: `register_credential_for.rs`.

Description: `sealed_payload: [u8; 128]` is stored as-is. No on-chain check that the buffer is well-formed. A buggy operator-api could submit a buffer of all zeros.

Risk: Bridges have their own well-formedness check (`isWellFormedSealed` in `sim/codec/sealing.ts` rejects all-zero ephemeral pubkeys). A malformed credential would propagate to RADIUS as a non-decryptable entry — the bridge alerts but doesn't block.

Recommendation: Add a simple "first 32 bytes must not be all zero" check in the handler. Defensive, cheap (4-line change), prevents the "oops we wrote a default-init buffer" failure mode entirely.

3.16 LOW — L-02: Negative-path test coverage could be denser

Location: `sim/scenario-mpsk-production.ts` has one negative test (stale Registrar attempting to mint after revocation).

Description: Each error code in `error.rs` should have at least one bankrun scenario that hits it. Currently uncovered (or sparsely covered): `EmptyAccessDomainName`, `AccessDomainNameTooLong`, `InvalidControlPlaneDevice`, `DomainAuthorityAlreadyExpired`, `DomainAuthorityWrongRole`, several PSK validation paths.

Recommendation: Add `sim/scenario-negative-paths.ts` exercising each error code at least once. Useful pre-audit (proves the error codes are wired correctly) and pays off long-term as regression coverage.

3.17 LOW — L-03: lat/lon scaling not explicitly documented

Location: `state/device_location.rs`.

Description: `latitude: i64` and `longitude: i64` — no comment on scale (degrees $\times 10^5$? $\times 10^6$? $\times 10^7$?). The scenario script uses `37774900n` for latitude, suggesting $\times 10^5$ ($37.77490^\circ = \text{SF}$). But this is implicit.

Recommendation: Add a doc comment specifying the scale. E.g., `/// Geographic position - latitude (degrees scaled by 1e5)`. Auditors and SDK authors both benefit.

3.18 LOW — L-04: `set_access_domain_gateway_device` accepts arbitrary pubkey

Location: `app/access_domain/set_gateway_device.rs`.

Description: Unlike `set_control_plane_device`, this instruction does NOT reject `Pubkey::default()`. Setting `gateway_device = Some(Pubkey::default())` is technically allowed.

Risk: Trivial — there's no semantic difference between `gateway_device = None` and `gateway_device = Some(default())`. Off-chain consumers may handle these inconsistently.

Recommendation: Either reject `Some(default())` (treat it as `None` at the contract level) or document that the two are indistinguishable.

3.19 INFO — I-01: Account size summary

For the auditor's reference (Anchor `#[derive(InitSpace)]` budgets):

Account	Bytes (incl. 8-byte disc)	Notes
<code>Config</code>	49	minimal singleton
<code>AccessDomain</code>	200	with all optionals + 32-byte name
<code>AuthMethod</code>	338	parameters[256] dominant
<code>Credential</code>	312	sealed_payload[128] dominant
<code>DomainAuthority</code>	160	with 32-byte label + expiry
<code>DeviceModel</code>	215	with 64-byte mfg/model strings
<code>Device</code>	119	
<code>DeviceLocation</code>	73	
<code>LocalDomain</code>	87	

All within reasonable rent budgets (max ~ 0.0024 SOL for the largest account at current rates).

3.20 INFO — I-02: Off-chain dependencies

The off-chain sealing layer (`sim/codec/sealing.ts`) depends on:

- `tweetnacl` (well-vetted libsodium-compatible JS port)
- `tweetnacl-sealedbox-js` (sealed-box specific)
- `@noble/curves` for Ed25519↔Curve25519 conversion (cryptographically rigorous, audited)

The on-chain program is independent of these — it only stores opaque bytes. Auditor needs only to confirm that the program does not attempt to interpret `sealed_payload`.

4. Recommended pre-external-audit punchlist

Before sending this branch to a third-party audit firm, recommend addressing in this order:

1. **H-01** — add `set_access_domain_owner` (~ 30 lines)
2. **H-02** — add `revoke_auth_method` (~ 40 lines)
3. **H-03** — fix or delete the dead-code height check (~ 2 lines)

4. **H-04** — decide on (a) require beneficiary signature OR (b) add `accept_device_assignment` two-step. Document the choice. (~50 lines for option b)
5. **M-05** — delete dead helpers in `utils/`. (~delete 3 functions)
6. **M-06** — add seed-byte stability unit tests. (~20 lines)
7. **M-09** — document the seed/owner divergence note. (no code)

Estimated effort: ~150 lines of code + tests, 1–2 days of work.

After that the program is in good shape for a \$15–30K external audit (slim Anchor program, 16 instructions, no SPL token CPI, no Raydium / external program CPI). Recommended firms: Halborn, Neodyme, OtterSec, Sec3.

The MEDIUM and LOW items can be addressed in conversation with the auditor or in a v2.2.0 follow-up.

5. Out of scope

Out of scope for this internal review:

- Off-chain operator-api implementation (sealing layer, retry logic, key rotation). The protocol contract (`docs/sot-bridge-protocol.md`) documents the requirements; the operator-api is not yet written.
- Bridge implementation correctness. Bridge is in the team's separate repository; correctness is verified end-to-end by the `devnet-mpsk-*scenario.ts` E2E tests.
- TS SDK security. The SDK lives in `sdk/` and is not part of the on-chain attack surface.
- Multisig configuration / key custody best practices. These are operational and addressed in the deployment runbook.

Known limitations (by design, not findings):

- Single-program lean build. The integrated branch (`access-domain-redesign-integration`) brings in the commercial layer; it's a separate audit scope.
 - No built-in multi-signature requirement. Mainnet deployments expected to use Squads or similar at the wallet level.
 - No on-chain customer enrollment authorization (the protocol assumes operator-side customer onboarding via the operator-api; on-chain we just see the resulting Credential mint).
-

6. Sign-off

This review covers all 27 source files in the lean program at program version 2.1.0 (devnet program ID `rHSumT63fgwAsHhKbR39AijjY28H99VuvM8xNaNehkj` , deployed at slot 461303727).

Status: ready for external audit after the H-01..H-04 punchlist.

Document generated as part of pre-audit prep, internal-only. Should accompany the program source when handed to the external audit firm.